

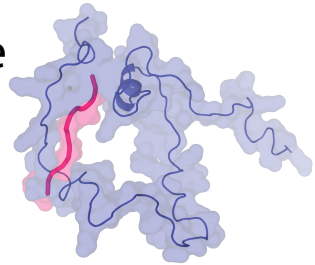
CIS 6270

Module 1: Introduction and Math Foundations

Lecture 1.4: Vector Fields, Probability Conservation, and Optimization
Thu, Sep 3, 2026

Pranam Chatterjee, PhD
Assistant Professor of Bioengineering
Assistant Professor of Computer and Information Science
University of Pennsylvania

Penn Engineering
University of Pennsylvania



The positive curvature in one direction cancels the negative curvature in the other.

Let

$$f(x_1, x_2) = x_1^2 + 3x_2^2$$

The second derivatives are

$$\frac{\partial^2 f}{\partial x_1^2} = 2, \qquad \frac{\partial^2 f}{\partial x_2^2} = 6$$

Therefore,

$$\Delta f = 2 + 6 = 8$$

Now consider,

$$g(x_1, x_2) = x_1^2 - x_2^2$$

Then,

$$\Delta g = 2 - 2 = 0$$

The simplest equation describing diffusion

$$\boxed{\frac{\partial p_t(x)}{\partial t} = D\Delta p_t(x)}$$

The simplest equation describing diffusion

$$\frac{\partial p_t(x)}{\partial t} = D\Delta p_t(x)$$

The time derivative $\partial p_t(x)/\partial t$ of the density equals the diffusion coefficient D multiplied by the Laplacian $\Delta p_t(x)$, causing density to flow away from sharp local peaks and into nearby lower-density regions.

The simplest equation describing diffusion

$$\frac{\partial p_t(x)}{\partial t} = D\Delta p_t(x)$$

The time derivative $\partial p_t(x)/\partial t$ of the density equals the diffusion coefficient D multiplied by the Laplacian $\Delta p_t(x)$, causing density to flow away from sharp local peaks and into nearby lower-density regions.

At a sharp density peak, the Laplacian is negative, so density decreases.

The simplest equation describing diffusion

$$\frac{\partial p_t(x)}{\partial t} = D\Delta p_t(x)$$

The time derivative $\partial p_t(x)/\partial t$ of the density equals the diffusion coefficient D multiplied by the Laplacian $\Delta p_t(x)$, causing density to flow away from sharp local peaks and into nearby lower-density regions.

At a sharp density peak, the Laplacian is negative, so density decreases.

In a nearby valley, the Laplacian is positive, so density increases.

The simplest equation describing diffusion

$$\frac{\partial p_t(x)}{\partial t} = D\Delta p_t(x)$$

The time derivative $\partial p_t(x)/\partial t$ of the density equals the diffusion coefficient D multiplied by the Laplacian $\Delta p_t(x)$, causing density to flow away from sharp local peaks and into nearby lower-density regions.

At a sharp density peak, the Laplacian is negative, so density decreases.

In a nearby valley, the Laplacian is positive, so density increases.

The distribution becomes smoother over time.

A stochastic process can contain directed motion and random diffusion

$$dX_t = b_t(X_t)\,dt + \sigma_t\,dW_t$$

A stochastic process can contain directed motion and random diffusion

$$dX_t = b_t(X_t)\,dt + \sigma_t\,dW_t$$

The infinitesimal change dX_t in the random state X_t equals the drift vector $b_t(X_t)$ multiplied by the small time increment dt plus Gaussian Brownian noise dW_t scaled by the noise magnitude σ_t.

A stochastic process can contain directed motion and random diffusion

$$dX_t = b_t(X_t)\,dt + \sigma_t\,dW_t$$

The infinitesimal change dX_t in the random state X_t equals the drift vector $b_t(X_t)$ multiplied by the small time increment dt plus Gaussian Brownian noise dW_t scaled by the noise magnitude σ_t.

Its density evolves according to

$$\boxed{\frac{\partial p_t}{\partial t} = -\nabla \cdot (b_t p_t) + \frac{\sigma_t^2}{2}\Delta p_t}$$

A stochastic process can contain directed motion and random diffusion

$$dX_t = b_t(X_t)\,dt + \sigma_t\,dW_t$$

The infinitesimal change dX_t in the random state X_t equals the drift vector $b_t(X_t)$ multiplied by the small time increment dt plus Gaussian Brownian noise dW_t scaled by the noise magnitude σ_t.

Its density evolves according to

$$\boxed{\frac{\partial p_t}{\partial t} = -\nabla \cdot (b_t p_t) + \frac{\sigma_t^2}{2}\Delta p_t}$$

The time derivative of density p_t equals a transport term given by the negative divergence of the drift flux $b_t p_t$ plus a diffusion term given by one half σ_t squared times the Laplacian of p_t.

A stochastic process can contain directed motion and random diffusion

$$dX_t = b_t(X_t)\,dt + \sigma_t\,dW_t$$

The infinitesimal change dX_t in the random state X_t equals the drift vector $b_t(X_t)$ multiplied by the small time increment dt plus Gaussian Brownian noise dW_t scaled by the noise magnitude σ_t.

Its density evolves according to

$$\frac{\partial p_t}{\partial t} = -\nabla \cdot (b_t p_t) + \frac{\sigma_t^2}{2}\Delta p_t$$

The first term moves density through directed drift.

A stochastic process can contain directed motion and random diffusion

$$dX_t = b_t(X_t)\,dt + \sigma_t\,dW_t$$

The infinitesimal change dX_t in the random state X_t equals the drift vector $b_t(X_t)$ multiplied by the small time increment dt plus Gaussian Brownian noise dW_t scaled by the noise magnitude σ_t.

Its density evolves according to

$$\frac{\partial p_t}{\partial t} = -\nabla \cdot (b_t p_t) + \frac{\sigma_t^2}{2}\Delta p_t$$

The first term moves density through directed drift. The second term spreads density through random noise.

This is the Fokker-Planck equation for isotropic diffusion

$$dX_t = b_t(X_t)\,dt + \sigma_t\,dW_t$$

The infinitesimal change dX_t in the random state X_t equals the drift vector $b_t(X_t)$ multiplied by the small time increment dt plus Gaussian Brownian noise dW_t scaled by the noise magnitude σ_t.

Its density evolves according to

$$\frac{\partial p_t}{\partial t} = -\nabla \cdot (b_t p_t) + \frac{\sigma_t^2}{2}\Delta p_t$$

The first term moves density through directed drift. The second term spreads density through random noise.

Now everything has a nice role!

Now everything has a nice role!

- The gradient describes how a scalar score or density changes.

Now everything has a nice role!

- The gradient describes how a scalar score or density changes.
- The score is the gradient of log density.

Now everything has a nice role!

- The gradient describes how a scalar score or density changes.
- The score is the gradient of log density.
- A vector field determines sample velocity.

Now everything has a nice role!

- The gradient describes how a scalar score or density changes.
- The score is the gradient of log density.
- A vector field determines sample velocity.
- Divergence measures local expansion and log-density change.

Now everything has a nice role!

- The gradient describes how a scalar score or density changes.
- The score is the gradient of log density.
- A vector field determines sample velocity.
- Divergence measures local expansion and log-density change.
- The Laplacian describes diffusion and density smoothing.

Now everything has a nice role!

- The gradient describes how a scalar score or density changes.
- The score is the gradient of log density.
- A vector field determines sample velocity.
- Divergence measures local expansion and log-density change.
- The Laplacian describes diffusion and density smoothing.
- The Fokker-Planck equation combines directed transport and random diffusion.

Now everything has a nice role!

- The gradient describes how a scalar score or density changes.
- The score is the gradient of log density.
- A vector field determines sample velocity.
- Divergence measures local expansion and log-density change.
- The Laplacian describes diffusion and density smoothing.
- The Fokker-Planck equation combines directed transport and random diffusion.

A probability distribution tells us how mass is allocated.

A probability distribution tells us how mass is allocated.

We now need numerical quantities that answer three related questions:

A probability distribution tells us how mass is allocated.

We now need numerical quantities that answer three related questions:

1. How uncertain is one distribution?

A probability distribution tells us how mass is allocated.

We now need numerical quantities that answer three related questions:

1. How uncertain is one distribution?
2. How surprised are we when one distribution scores samples from another?

A probability distribution tells us how mass is allocated.

We now need numerical quantities that answer three related questions:

1. How uncertain is one distribution?
2. How surprised are we when one distribution scores samples from another?
3. How different are two distributions?

A probability distribution tells us how mass is allocated.

We now need numerical quantities that answer three related questions:

1. How uncertain is one distribution?
2. How surprised are we when one distribution scores samples from another?
3. How different are two distributions?

The starting point is the information carried by one observation.

The information associated with observing x is:

$$I(x) = -\log p(x)$$

The information associated with observing x is:

$$I(x) = -\log p(x)$$

The information I(x) associated with observing outcome x equals the negative logarithm of its probability p(x), so an outcome with lower probability carries greater information when it occurs.

An example

If

$$p(x) = 0.5$$

An example

If

$$p(x) = 0.5$$

Then

$$I(x) = -\log(0.5) \approx 0.693$$

An example

If

$$p(x) = 0.5$$

Then

$$I(x) = -\log(0.5) \approx 0.693$$

An example

If

$$p(x) = 0.5$$

Then

$$I(x) = -\log(0.5) \approx 0.693$$

But If

$$p(x) = 0.01$$

An example

If

$$p(x) = 0.5$$

Then

$$I(x) = -\log(0.5) \approx 0.693$$

But If

$$p(x) = 0.01$$

Then

$$I(x) = -\log(0.01) \approx 4.605$$

The event with probability 0.01 is more surprising and therefore carries more information.

Information content describes one observed outcome.

Information content describes one observed outcome.

Averaging this quantity across all possible outcomes gives the uncertainty of the complete distribution.

Entropy

Entropy

Entropy is the expected information under the distribution itself

$$H(p) = -\sum_{x \in \mathcal{X}} p(x) \log p(x)$$

Entropy

Entropy is the expected information under the distribution itself

$$H(p) = -\sum_{x \in \mathcal{X}} p(x) \log p(x)$$

The entropy H(p) equals the negative sum over every state x in the state space $\mathcal{X}$ of its probability p(x) multiplied by log p(x), giving the expected information obtained from one sample drawn from p.

Entropy

Entropy is the expected information under the distribution itself

$$H(p) = -\sum_{x \in \mathcal{X}} p(x) \log p(x)$$

The entropy H(p) equals the negative sum over every state x in the state space $\mathcal{X}$ of its probability p(x) multiplied by log p(x), giving the expected information obtained from one sample drawn from p.

- High entropy means probability is distributed across many states.

Entropy

Entropy is the expected information under the distribution itself

$$H(p) = -\sum_{x \in \mathcal{X}} p(x) \log p(x)$$

The entropy H(p) equals the negative sum over every state x in the state space $\mathcal{X}$ of its probability p(x) multiplied by log p(x), giving the expected information obtained from one sample drawn from p.

- High entropy means probability is distributed across many states.
- Low entropy means probability is concentrated on a few states.

Entropy

Entropy is the expected information under the distribution itself

$$H(p) = -\sum_{x \in \mathcal{X}} p(x) \log p(x)$$

The entropy H(p) equals the negative sum over every state x in the state space $\mathcal{X}$ of its probability p(x) multiplied by log p(x), giving the expected information obtained from one sample drawn from p.

- High entropy means probability is distributed across many states.
- Low entropy means probability is concentrated on a few states.
- A deterministic distribution has entropy zero.

Entropy

Entropy is the expected information under the distribution itself

$$H(p) = -\sum_{x \in \mathcal{X}} p(x) \log p(x)$$

The entropy H(p) equals the negative sum over every state x in the state space $\mathcal{X}$ of its probability p(x) multiplied by log p(x), giving the expected information obtained from one sample drawn from p.

- High entropy means probability is distributed across many states.
- Low entropy means probability is concentrated on a few states.
- A deterministic distribution has entropy zero.
- A uniform distribution over K states has entropy log K.

A short example

For

$$p = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix}$$

A short example

For

$$p = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix}$$

the entropy is

$$H(p) = -0.5 \log(0.5) - 0.5 \log(0.5)$$

A short example

For

$$p = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix}$$

the entropy is

$$\begin{aligned} H(p) &= -0.5 \log(0.5) - 0.5 \log(0.5) \\ &= \log 2 \end{aligned}$$

A short example

For

$$p = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix}$$

the entropy is

$$\begin{aligned} H(p) &= -0.5 \log(0.5) - 0.5 \log(0.5) \\ &= \log 2 \\ &\approx 0.693. \end{aligned}$$

A short example

For

$$p = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix}$$

the entropy is

$$\begin{aligned} H(p) &= -0.5 \log(0.5) - 0.5 \log(0.5) \\ &= \log 2 \\ &\approx 0.693. \end{aligned}$$

For

$$q = \begin{bmatrix} 0.9 \\ 0.1 \end{bmatrix}$$

A short example

For

$$p = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix}$$

the entropy is

$$\begin{aligned} H(p) &= -0.5 \log(0.5) - 0.5 \log(0.5) \\ &= \log 2 \\ &\approx 0.693. \end{aligned}$$

For

$$q = \begin{bmatrix} 0.9 \\ 0.1 \end{bmatrix}$$

the entropy is

$$H(q) = -0.9 \log(0.9) - 0.1 \log(0.1) \approx 0.325$$

The balanced distribution has greater uncertainty because neither outcome is strongly preferred.

the entropy is

$$\begin{aligned} H(p) &= -0.5 \log(0.5) - 0.5 \log(0.5) \\ &= \log 2 \\ &\approx 0.693. \end{aligned}$$

For

$$q = \begin{bmatrix} 0.9 \\ 0.1 \end{bmatrix}$$

the entropy is

$$H(q) = -0.9 \log(0.9) - 0.1 \log(0.1) \approx 0.325$$

This definition uses a finite sum.

the entropy is

$$\begin{aligned} H(p) &= -0.5\log(0.5) - 0.5\log(0.5) \\ &= \log 2 \\ &\approx 0.693. \end{aligned}$$

For

$$q = \begin{bmatrix} 0.9 \\ 0.1 \end{bmatrix}$$

the entropy is

$$H(q) = -0.9\log(0.9) - 0.1\log(0.1) \approx 0.325$$

But continuous densities need the corresponding integral.

the entropy is

$$\begin{aligned} H(p) &= -0.5\log(0.5) - 0.5\log(0.5) \\ &= \log 2 \\ &\approx 0.693. \end{aligned}$$

For

$$q = \begin{bmatrix} 0.9 \\ 0.1 \end{bmatrix}$$

the entropy is

$$H(q) = -0.9\log(0.9) - 0.1\log(0.1) \approx 0.325$$

Continuous entropy

Continuous entropy

For a continuous density,

$$h(p) = -\int p(x) \log p(x)\, dx$$

Continuous entropy

For a continuous density,

$$h(p) = -\int p(x) \log p(x)\, dx$$

The differential entropy h(p) equals the negative integral over all continuous states x of the density p(x) multiplied by its log density log p(x).

Continuous entropy

For a continuous density,

$$h(p) = -\int p(x) \log p(x)\, dx$$

The differential entropy h(p) equals the negative integral over all continuous states x of the density p(x) multiplied by its log density log p(x).

Differential entropy can be negative and changes under coordinate transformations.

Continuous entropy

For a continuous density,

$$h(p) = -\int p(x) \log p(x)\, dx$$

The differential entropy h(p) equals the negative integral over all continuous states x of the density p(x) multiplied by its log density log p(x).

Differential entropy can be negative and changes under coordinate transformations.

These properties arise because a density depends on the units and coordinates used to describe space.

Entropy measures uncertainty using the same distribution for sampling and scoring.

Entropy measures uncertainty using the same distribution for sampling and scoring.

Model training often involves samples from one distribution and probabilities assigned by another.

Entropy measures uncertainty using the same distribution for sampling and scoring.

Model training often involves samples from one distribution and probabilities assigned by another.

This leads to cross-entropy.

Cross-entropy

Cross-entropy

Suppose samples follow p while a model assigns probabilities according to q. Their cross-entropy is:

$$H(p, q) = -\sum_{x \in \mathcal{X}} p(x) \log q(x)$$

Cross-entropy

Suppose samples follow p while a model assigns probabilities according to q. Their cross-entropy is:

$$H(p, q) = -\sum_{x \in \mathcal{X}} p(x) \log q(x)$$

The cross-entropy H(p,q) equals the expected negative log probability assigned by the scoring distribution q to an outcome x drawn from the sampling distribution p.

Short example

Let

$$p = \begin{bmatrix} 0.8 \\ 0.2 \end{bmatrix}, \qquad q = \begin{bmatrix} 0.6 \\ 0.4 \end{bmatrix}$$

Short example

Let

$$p = \begin{bmatrix} 0.8 \\ 0.2 \end{bmatrix}, \qquad q = \begin{bmatrix} 0.6 \\ 0.4 \end{bmatrix}$$

Then

$$\begin{aligned} H(p, q) &= -0.8 \log(0.6) - 0.2 \log(0.4) \\ &\approx 0.592. \end{aligned}$$

Short example

Let

$$p = \begin{bmatrix} 0.8 \\ 0.2 \end{bmatrix}, \qquad q = \begin{bmatrix} 0.6 \\ 0.4 \end{bmatrix}$$

Then

$$\begin{aligned} H(p, q) &= -0.8 \log(0.6) - 0.2 \log(0.4) \\ &\approx 0.592. \end{aligned}$$

The entropy of p is

$$H(p) = -0.8 \log(0.8) - 0.2 \log(0.2) \approx 0.500$$

Scoring samples from p using q requires approximately 0.592 nats (units of entropy) on average.

Let

$$p = \begin{bmatrix} 0.8 \\ 0.2 \end{bmatrix}, \qquad q = \begin{bmatrix} 0.6 \\ 0.4 \end{bmatrix}$$

Then

$$\begin{aligned} H(p, q) &= -0.8 \log(0.6) - 0.2 \log(0.4) \\ &\approx 0.592. \end{aligned}$$

The entropy of p is

$$H(p) = -0.8 \log(0.8) - 0.2 \log(0.2) \approx 0.500$$

Scoring them using the correct distribution p requires only 0.500 nats.

Let

$$p = \begin{bmatrix} 0.8 \\ 0.2 \end{bmatrix}, \qquad q = \begin{bmatrix} 0.6 \\ 0.4 \end{bmatrix}$$

Then

$$\begin{aligned} H(p, q) &= -0.8 \log(0.6) - 0.2 \log(0.4) \\ &\approx 0.592. \end{aligned}$$

The entropy of p is

$$H(p) = -0.8 \log(0.8) - 0.2 \log(0.2) \approx 0.500$$

The additional 0.092 nats comes from the mismatch between p and q.

Let

$$p = \begin{bmatrix} 0.8 \\ 0.2 \end{bmatrix}, \qquad q = \begin{bmatrix} 0.6 \\ 0.4 \end{bmatrix}$$

Then

$$\begin{aligned} H(p, q) &= -0.8 \log(0.6) - 0.2 \log(0.4) \\ &\approx 0.592. \end{aligned}$$

The entropy of p is

$$H(p) = -0.8 \log(0.8) - 0.2 \log(0.2) \approx 0.500$$

We use KL divergence to isolate this mismatch.

Let

$$p = \begin{bmatrix} 0.8 \\ 0.2 \end{bmatrix}, \qquad q = \begin{bmatrix} 0.6 \\ 0.4 \end{bmatrix}$$

Then

$$\begin{aligned} H(p, q) &= -0.8 \log(0.6) - 0.2 \log(0.4) \\ &\approx 0.592. \end{aligned}$$

The entropy of p is

$$H(p) = -0.8 \log(0.8) - 0.2 \log(0.2) \approx 0.500$$

Kullback–Leibler (KL) divergence

Kullback–Leibler (KL) divergence

The KL divergence from p to q is

$$D_{\mathrm{KL}}(p\|q) = \sum_{x\in\mathcal{X}} p(x)\log\frac{p(x)}{q(x)}$$

Kullback–Leibler (KL) divergence

The KL divergence from p to q is

$$D_{\mathrm{KL}}(p\|q) = \sum_{x\in\mathcal{X}} p(x)\log\frac{p(x)}{q(x)}$$

The KL divergence D_KL(p || q) equals the expected log ratio between the reference probability p(x) and the comparison probability q(x) for outcomes x sampled from p, measuring the discrepancy created when q is used in place of p.

Kullback–Leibler (KL) divergence

The KL divergence from p to q is

$$D_{\mathrm{KL}}(p\|q) = \sum_{x\in\mathcal{X}} p(x)\log\frac{p(x)}{q(x)}$$

The KL divergence D_KL(p || q) equals the expected log ratio between the reference probability p(x) and the comparison probability q(x) for outcomes x sampled from p, measuring the discrepancy created when q is used in place of p.

If q(x)=0 for a state where p(x)>0, then the KL divergence is infinite. The comparison distribution has assigned zero probability to an event that can occur under the reference distribution.

A short example

Using

$$p = \begin{bmatrix} 0.8 \\ 0.2 \end{bmatrix}, \qquad q = \begin{bmatrix} 0.6 \\ 0.4 \end{bmatrix}$$

A short example

Using

$$p = \begin{bmatrix} 0.8 \\ 0.2 \end{bmatrix}, \qquad q = \begin{bmatrix} 0.6 \\ 0.4 \end{bmatrix}$$

We obtain

$$\begin{aligned} D_{\mathrm{KL}}(p\|q) &= 0.8 \log \frac{0.8}{0.6} + 0.2 \log \frac{0.2}{0.4} \\ &\approx 0.0915. \end{aligned}$$

A short example

Using

$$p = \begin{bmatrix} 0.8 \\ 0.2 \end{bmatrix}, \qquad q = \begin{bmatrix} 0.6 \\ 0.4 \end{bmatrix}$$

We obtain

$$\begin{aligned} D_{\mathrm{KL}}(p\|q) &= 0.8 \log \frac{0.8}{0.6} + 0.2 \log \frac{0.2}{0.4} \\ &\approx 0.0915. \end{aligned}$$

A value of zero would mean that q matches p exactly.

A short example

Using

$$p = \begin{bmatrix} 0.8 \\ 0.2 \end{bmatrix}, \qquad q = \begin{bmatrix} 0.6 \\ 0.4 \end{bmatrix}$$

We obtain

$$\begin{aligned} D_{\mathrm{KL}}(p\|q) &= 0.8 \log \frac{0.8}{0.6} + 0.2 \log \frac{0.2}{0.4} \\ &\approx 0.0915. \end{aligned}$$

A value of zero would mean that q matches p exactly.

The numerical value closely matches the difference between cross-entropy and entropy. This relationship holds for every pair of distributions.

Let’s expand the probability ratio

$$D_{\mathrm{KL}}(p\|q) = \sum_{x} p(x)\,[\log p(x) - \log q(x)]$$

Let’s expand the probability ratio

$$
\begin{aligned}
D_{\mathrm{KL}}(p\|q) &= \sum_{x} p(x)\,[\log p(x) - \log q(x)] \\
&= \sum_{x} p(x) \log p(x) - \sum_{x} p(x) \log q(x).
\end{aligned}
$$

Let’s expand the probability ratio

$$
\begin{aligned}
D_{\mathrm{KL}}(p\|q) &= \sum_x p(x)\,[\log p(x) - \log q(x)] \\
&= \sum_x p(x) \log p(x) - \sum_x p(x) \log q(x).
\end{aligned}
$$

Recognize the entropy and cross-entropy terms:

$$
D_{\mathrm{KL}}(p\|q) = -H(p) + H(p,q)
$$

Let’s expand the probability ratio

$$
\begin{aligned}
D_{\mathrm{KL}}(p\|q) &= \sum_x p(x)\,[\log p(x) - \log q(x)] \\
&= \sum_x p(x)\log p(x) - \sum_x p(x)\log q(x).
\end{aligned}
$$

Recognize the entropy and cross-entropy terms:

$$
D_{\mathrm{KL}}(p\|q) = -H(p) + H(p,q)
$$

Therefore,

$$
\boxed{H(p,q) = H(p) + D_{\mathrm{KL}}(p\|q)}
$$

Let’s expand the probability ratio

$$
\begin{aligned}
D_{\mathrm{KL}}(p\|q) &= \sum_x p(x)\,[\log p(x) - \log q(x)] \\
&= \sum_x p(x)\log p(x) - \sum_x p(x)\log q(x).
\end{aligned}
$$

Recognize the entropy and cross-entropy terms:

$$D_{\mathrm{KL}}(p\|q) = -H(p) + H(p,q)$$

Therefore,

$$\boxed{H(p,q) = H(p) + D_{\mathrm{KL}}(p\|q)}$$

The cross-entropy H(p,q) equals the intrinsic entropy H(p) of the sampling distribution p plus the KL divergence $D_{KL}(p\|q)$ caused by the mismatch between p and q.

Let’s expand the probability ratio

$$
\begin{aligned}
D_{\mathrm{KL}}(p\|q) &= \sum_x p(x)\,[\log p(x) - \log q(x)] \\
&= \sum_x p(x)\log p(x) - \sum_x p(x)\log q(x).
\end{aligned}
$$

Recognize the entropy and cross-entropy terms:

$$D_{\mathrm{KL}}(p\|q) = -H(p) + H(p,q)$$

Therefore,

$$\boxed{H(p,q) = H(p) + D_{\mathrm{KL}}(p\|q)}$$

The entropy H(p) is fixed once the data distribution is fixed. The KL term is the additional cost that can be reduced by improving q.

For this interpretation to make sense, the KL term must always be nonnegative

For this interpretation to make sense, the KL term must always be nonnegative

Use the inequality

$$\log u \leq u - 1$$

For this interpretation to make sense, the KL term must always be nonnegative

Use the inequality

$$\log u \leq u - 1$$

Then

$$-D_{\mathrm{KL}}(p\|q) = \sum_x p(x) \log \frac{q(x)}{p(x)}$$

For this interpretation to make sense, the KL term must always be nonnegative

Use the inequality

$$\log u \leq u - 1$$

Then

$$\begin{aligned} -D_{\mathrm{KL}}(p\|q) &= \sum_x p(x) \log \frac{q(x)}{p(x)} \\ &\leq \sum_x p(x) \left(\frac{q(x)}{p(x)} - 1 \right) \end{aligned}$$

For this interpretation to make sense, the KL term must always be nonnegative

Use the inequality

$$\log u \leq u - 1$$

Then

$$\begin{aligned} -D_{\mathrm{KL}}(p\|q) &= \sum_x p(x) \log \frac{q(x)}{p(x)} \\ &\leq \sum_x p(x) \left(\frac{q(x)}{p(x)} - 1 \right) \\ &= \sum_x q(x) - \sum_x p(x) \end{aligned}$$

For this interpretation to make sense, the KL term must always be nonnegative

Use the inequality

$$\log u \leq u - 1$$

Then

$$\begin{aligned} -D_{\mathrm{KL}}(p\|q) &= \sum_x p(x) \log \frac{q(x)}{p(x)} \\ &\leq \sum_x p(x) \left(\frac{q(x)}{p(x)} - 1 \right) \\ &= \sum_x q(x) - \sum_x p(x) \\ &= 1 - 1 \\ &= 0. \end{aligned}$$

For this interpretation to make sense, the KL term must always be nonnegative

Use the inequality

$$\log u \leq u - 1$$

Then

$$\begin{aligned} -D_{\mathrm{KL}}(p\|q) &= \sum_x p(x) \log \frac{q(x)}{p(x)} \\ &\leq \sum_x p(x) \left(\frac{q(x)}{p(x)} - 1 \right) \\ &= \sum_x q(x) - \sum_x p(x) \\ &= 1 - 1 \\ &= 0. \end{aligned}$$

Therefore:

$$\boxed{D_{\mathrm{KL}}(p\|q) \geq 0}$$

The KL divergence D_KL(p‖q) is nonnegative because the logarithm inequality bounds its negative by zero, with equality only when p and q agree on every state carrying probability under p.

For this interpretation to make sense, the KL term must always be nonnegative

Use the inequality

$$\log u \leq u - 1$$

Then

$$\begin{aligned} -D_{\mathrm{KL}}(p\|q) &= \sum_x p(x) \log \frac{q(x)}{p(x)} \\ &\leq \sum_x p(x) \left(\frac{q(x)}{p(x)} - 1 \right) \\ &= \sum_x q(x) - \sum_x p(x) \\ &= 1 - 1 \\ &= 0. \end{aligned}$$

Therefore:

$$\boxed{D_{\mathrm{KL}}(p\|q) \geq 0}$$

Nonnegativity allows KL divergence to quantify discrepancy.

For this interpretation to make sense, the KL term must always be nonnegative

Use the inequality

$$\log u \leq u - 1$$

Then

$$\begin{aligned} -D_{\mathrm{KL}}(p\|q) &= \sum_x p(x) \log \frac{q(x)}{p(x)} \\ &\leq \sum_x p(x) \left(\frac{q(x)}{p(x)} - 1 \right) \\ &= \sum_x q(x) - \sum_x p(x) \\ &= 1 - 1 \\ &= 0. \end{aligned}$$

Therefore:

$$\boxed{D_{\mathrm{KL}}(p\|q) \geq 0}$$

Nonnegativity allows KL divergence to quantify discrepancy.

Its direction still matters because the expectation is taken under the first distribution.

KL divergence is asymmetric

Generally,

$$D_{\mathrm{KL}}(p\|q) \neq D_{\mathrm{KL}}(q\|p)$$

KL divergence is asymmetric

Generally,

$$D_{\mathrm{KL}}(p\|q) \neq D_{\mathrm{KL}}(q\|p)$$

$$p = \begin{bmatrix} 0.8 \\ 0.2 \end{bmatrix}, \qquad q = \begin{bmatrix} 0.6 \\ 0.4 \end{bmatrix}$$

KL divergence is asymmetric

Generally,

$$D_{\mathrm{KL}}(p\|q) \neq D_{\mathrm{KL}}(q\|p)$$

$$p = \begin{bmatrix} 0.8 \\ 0.2 \end{bmatrix}, \qquad q = \begin{bmatrix} 0.6 \\ 0.4 \end{bmatrix}$$

$$\begin{aligned} D_{\mathrm{KL}}(p\|q) &= 0.8 \log \frac{0.8}{0.6} + 0.2 \log \frac{0.2}{0.4} \\ &\approx 0.0915. \end{aligned}$$

KL divergence is asymmetric

Generally,

$$D_{\mathrm{KL}}(p\|q) \neq D_{\mathrm{KL}}(q\|p)$$

$$p = \begin{bmatrix} 0.8 \\ 0.2 \end{bmatrix}, \qquad q = \begin{bmatrix} 0.6 \\ 0.4 \end{bmatrix}$$

$$\begin{aligned} D_{\mathrm{KL}}(p\|q) &= 0.8 \log \frac{0.8}{0.6} + 0.2 \log \frac{0.2}{0.4} \\ &\approx 0.0915. \end{aligned}$$

$$\begin{aligned} D_{\mathrm{KL}}(q\|p) &= 0.6 \log \frac{0.6}{0.8} + 0.4 \log \frac{0.4}{0.2} \\ &\approx 0.1046. \end{aligned}$$

KL divergence is asymmetric

Generally,

$$D_{\mathrm{KL}}(p\|q) \neq D_{\mathrm{KL}}(q\|p)$$

$$p = \begin{bmatrix} 0.8 \\ 0.2 \end{bmatrix}, \qquad q = \begin{bmatrix} 0.6 \\ 0.4 \end{bmatrix}$$

$$D_{\mathrm{KL}}(p\|q) = 0.8 \log \frac{0.8}{0.6} + 0.2 \log \frac{0.2}{0.4} \approx 0.0915.$$

$$D_{\mathrm{KL}}(q\|p) = 0.6 \log \frac{0.6}{0.8} + 0.4 \log \frac{0.4}{0.2} \approx 0.1046.$$

The KL divergence changes when p and q are reversed because $D_{KL}(p\|q)$ weights the log probability ratio using samples from p, while $D_{KL}(q\|p)$ weights it using samples from q.

KL divergence is asymmetric

Generally,

$$D_{\mathrm{KL}}(p\|q) \neq D_{\mathrm{KL}}(q\|p)$$

$$p = \begin{bmatrix} 0.8 \\ 0.2 \end{bmatrix}, \qquad q = \begin{bmatrix} 0.6 \\ 0.4 \end{bmatrix}$$

$$D_{\mathrm{KL}}(p\|q) = 0.8 \log \frac{0.8}{0.6} + 0.2 \log \frac{0.2}{0.4} \approx 0.0915.$$

$$D_{\mathrm{KL}}(q\|p) = 0.6 \log \frac{0.6}{0.8} + 0.4 \log \frac{0.4}{0.2} \approx 0.1046.$$

The first distribution determines which regions receive the most weight. This becomes important when choosing objectives for generative models.

KL divergence is asymmetric

Generally,

$$D_{\mathrm{KL}}(p\|q) \neq D_{\mathrm{KL}}(q\|p)$$

$$p = \begin{bmatrix} 0.8 \\ 0.2 \end{bmatrix}, \qquad q = \begin{bmatrix} 0.6 \\ 0.4 \end{bmatrix}$$

$$D_{\mathrm{KL}}(p\|q) = 0.8 \log \frac{0.8}{0.6} + 0.2 \log \frac{0.2}{0.4} \approx 0.0915.$$

$$D_{\mathrm{KL}}(q\|p) = 0.6 \log \frac{0.6}{0.8} + 0.4 \log \frac{0.4}{0.2} \approx 0.1046.$$

The most common training direction uses real data as the sampling distribution and the model as the scoring distribution. This produces maximum-likelihood training.

Maximum likelihood training

Maximum likelihood training

Let p_{data} be the data distribution and p_θ the model distribution.

Maximum likelihood training

Let p_{data} be the data distribution and p_θ the model distribution. The population negative log-likelihood is:

$$\mathcal{L}(\theta) = \mathbb{E}_{X \sim p_{\text{data}}} \left[-\log p_\theta(X) \right]$$

Maximum likelihood training

Let p_{data} be the data distribution and p_θ the model distribution. The population negative log-likelihood is:

$$\mathcal{L}(\theta) = \mathbb{E}_{X \sim p_{\text{data}}} \left[-\log p_\theta(X) \right]$$

The population loss $\mathcal{L}(\theta)$ equals the expected negative log probability assigned by the parameterized model p_θ to a random example X drawn from the data distribution p_{data}.

Maximum likelihood training

Let p_{data} be the data distribution and p_θ the model distribution. The population negative log-likelihood is:

$$\mathcal{L}(\theta) = \mathbb{E}_{X \sim p_{\text{data}}} \left[-\log p_\theta(X) \right]$$

The population loss $\mathcal{L}(\theta)$ equals the expected negative log probability assigned by the parameterized model p_θ to a random example X drawn from the data distribution p_{data}.

This is the cross-entropy $\mathcal{L}(\theta) = H(p_{\text{data}}, p_\theta)$

Maximum likelihood training

Let p_{data} be the data distribution and p_θ the model distribution. The population negative log-likelihood is:

$$\mathcal{L}(\theta) = \mathbb{E}_{X \sim p_{\text{data}}} \left[-\log p_\theta(X) \right]$$

The population loss $\mathcal{L}(\theta)$ equals the expected negative log probability assigned by the parameterized model p_θ to a random example X drawn from the data distribution p_{data}.

This is the cross-entropy $\mathcal{L}(\theta) = H(p_{\text{data}}, p_\theta)$

Using the earlier identity, $\mathcal{L}(\theta) = H(p_{\text{data}}) + D_{\text{KL}}(p_{\text{data}} \| p_\theta)$

Maximum likelihood training

Let p_{data} be the data distribution and p_θ the model distribution. The population negative log-likelihood is:

$$\mathcal{L}(\theta) = \mathbb{E}_{X \sim p_{\text{data}}} \left[-\log p_\theta(X) \right]$$

The population loss $\mathcal{L}(\theta)$ equals the expected negative log probability assigned by the parameterized model p_θ to a random example X drawn from the data distribution p_{data}.

This is the cross-entropy $\mathcal{L}(\theta) = H(p_{\text{data}}, p_\theta)$

Using the earlier identity, $\mathcal{L}(\theta) = H(p_{\text{data}}) + D_{\text{KL}}(p_{\text{data}} \| p_\theta)$

The entropy H(p_data) does not depend on the parameters θ. Therefore, minimizing negative log-likelihood also minimizes:

$$D_{\text{KL}}(p_{\text{data}} \| p_\theta)$$

Maximum likelihood training

Let p_{data} be the data distribution and p_θ the model distribution. The population negative log-likelihood is:

$$\mathcal{L}(\theta) = \mathbb{E}_{X \sim p_{\text{data}}} \left[-\log p_\theta(X) \right]$$

The population loss $\mathcal{L}(\theta)$ equals the expected negative log probability assigned by the parameterized model p_θ to a random example X drawn from the data distribution p_{data}.

This is the cross-entropy $\mathcal{L}(\theta) = H(p_{\text{data}}, p_\theta)$

Using the earlier identity, $\mathcal{L}(\theta) = H(p_{\text{data}}) + D_{\text{KL}}(p_{\text{data}} \| p_\theta)$

With a finite dataset, the expectation becomes

$$\boxed{\widehat{\mathcal{L}}(\theta) = -\frac{1}{N} \sum_{i=1}^{N} \log p_\theta(x^{(i)})}$$

Maximum likelihood training

Let p_{data} be the data distribution and p_θ the model distribution. The population negative log-likelihood is:

$$\mathcal{L}(\theta) = \mathbb{E}_{X \sim p_{\text{data}}} \left[-\log p_\theta(X) \right]$$

The population loss $\mathcal{L}(\theta)$ equals the expected negative log probability assigned by the parameterized model p_θ to a random example X drawn from the data distribution p_{data}.

This is the cross-entropy $\mathcal{L}(\theta) = H(p_{\text{data}}, p_\theta)$

Using the earlier identity, $\mathcal{L}(\theta) = H(p_{\text{data}}) + D_{\text{KL}}(p_{\text{data}} \| p_\theta)$

With a finite dataset, the expectation becomes

$$\widehat{\mathcal{L}}(\theta) = -\frac{1}{N} \sum_{i=1}^{N} \log p_\theta(x^{(i)})$$

The empirical negative log-likelihood L(θ) equals the negative average over N training examples $x^{(i)}$ of the log probability assigned to each example by the parameterized model p^θ.

This explains why assigning high probability to observed data is equivalent to reducing a distributional discrepancy.

Let p_{data} be the data distribution and p_θ the model distribution. The population negative log-likelihood is:

$$\mathcal{L}(\theta) = \mathbb{E}_{X \sim p_{\text{data}}} \left[-\log p_\theta(X) \right]$$

The population loss $\mathcal{L}(\theta)$ equals the expected negative log probability assigned by the parameterized model p_θ to a random example X drawn from the data distribution p_{data}.

This is the cross-entropy $\mathcal{L}(\theta) = H(p_{\text{data}}, p_\theta)$

Using the earlier identity, $\mathcal{L}(\theta) = H(p_{\text{data}}) + D_{\text{KL}}(p_{\text{data}} \| p_\theta)$

With a finite dataset, the expectation becomes

$$\widehat{\mathcal{L}}(\theta) = -\frac{1}{N} \sum_{i=1}^{N} \log p_\theta(x^{(i)})$$

The empirical negative log-likelihood L(θ) equals the negative average over N training examples $x^{(i)}$ of the log probability assigned to each example by the parameterized model p^θ.

These quantities are consistent across generative models

These quantities are consistent across generative models

- Autoregressive models minimize token-level cross-entropy.

These quantities are consistent across generative models

- Autoregressive models minimize token-level cross-entropy.
- Normalizing flows maximize exact likelihood after accounting for Jacobian determinants.

These quantities are consistent across generative models

- Autoregressive models minimize token-level cross-entropy.
- Normalizing flows maximize exact likelihood after accounting for Jacobian determinants.
- Variational objectives contain KL divergences between approximate and target distributions.

These quantities are consistent across generative models

- Autoregressive models minimize token-level cross-entropy.
- Normalizing flows maximize exact likelihood after accounting for Jacobian determinants.
- Variational objectives contain KL divergences between approximate and target distributions.
- Diffusion derivations contain KL terms between forward and reverse transitions.

These quantities are consistent across generative models

- Autoregressive models minimize token-level cross-entropy.
- Normalizing flows maximize exact likelihood after accounting for Jacobian determinants.
- Variational objectives contain KL divergences between approximate and target distributions.
- Diffusion derivations contain KL terms between forward and reverse transitions.
- Discrete diffusion objectives often reduce to weighted cross-entropy.

These quantities are consistent across generative models

- Autoregressive models minimize token-level cross-entropy.
- Normalizing flows maximize exact likelihood after accounting for Jacobian determinants.
- Variational objectives contain KL divergences between approximate and target distributions.
- Diffusion derivations contain KL terms between forward and reverse transitions.
- Discrete diffusion objectives often reduce to weighted cross-entropy.
- Guidance changes sampling through property-dependent log probabilities or energies.

These quantities are consistent across generative models

- Autoregressive models minimize token-level cross-entropy.
- Normalizing flows maximize exact likelihood after accounting for Jacobian determinants.
- Variational objectives contain KL divergences between approximate and target distributions.
- Diffusion derivations contain KL terms between forward and reverse transitions.
- Discrete diffusion objectives often reduce to weighted cross-entropy.
- Guidance changes sampling through property-dependent log probabilities or energies.

Entropy measures uncertainty within a distribution. Cross-entropy measures how well one distribution predicts samples from another. KL divergence isolates the mismatch between them.

These quantities are consistent across generative models

- Autoregressive models minimize token-level cross-entropy.
- Normalizing flows maximize exact likelihood after accounting for Jacobian determinants.
- Variational objectives contain KL divergences between approximate and target distributions.
- Diffusion derivations contain KL terms between forward and reverse transitions.
- Discrete diffusion objectives often reduce to weighted cross-entropy.
- Guidance changes sampling through property-dependent log probabilities or energies.

These objectives tell us what quantity should be minimized.

These quantities are consistent across generative models

- Autoregressive models minimize token-level cross-entropy.
- Normalizing flows maximize exact likelihood after accounting for Jacobian determinants.
- Variational objectives contain KL divergences between approximate and target distributions.
- Diffusion derivations contain KL terms between forward and reverse transitions.
- Discrete diffusion objectives often reduce to weighted cross-entropy.
- Guidance changes sampling through property-dependent log probabilities or energies.

These objectives tell us what quantity should be minimized.

The final question is how to minimize an objective while satisfying constraints such as normalization, fixed expectations, or valid probability vectors.

Generative-model training requires choosing parameters that minimize a loss.

Generative-model training requires choosing parameters that minimize a loss.

Probability distributions introduce additional constraints:

Generative-model training requires choosing parameters that minimize a loss.

Probability distributions introduce additional constraints:

- Probabilities must be nonnegative.

Generative-model training requires choosing parameters that minimize a loss.

Probability distributions introduce additional constraints:

- Probabilities must be nonnegative.
- Probabilities must sum to one.

Generative-model training requires choosing parameters that minimize a loss.

Probability distributions introduce additional constraints:

- Probabilities must be nonnegative.
- Probabilities must sum to one.
- Generated samples may need specified biological properties.

Generative-model training requires choosing parameters that minimize a loss.

Probability distributions introduce additional constraints:

- Probabilities must be nonnegative.
- Probabilities must sum to one.
- Generated samples may need specified biological properties.
- Dynamical systems may need stability or conservation conditions.

Generative-model training requires choosing parameters that minimize a loss.

Probability distributions introduce additional constraints:

- Probabilities must be nonnegative.
- Probabilities must sum to one.
- Generated samples may need specified biological properties.
- Dynamical systems may need stability or conservation conditions.

We therefore need optimization with and without constraints.

Unconstrained optimization

An unconstrained problem has the form:

$$\min_{\theta} \mathcal{L}(\theta)$$

Unconstrained optimization

An unconstrained problem has the form:

$$\min_{\theta} \mathcal{L}(\theta)$$

The optimization problem minimizes the loss function $\mathcal{L}(\theta)$ over every allowed value of the parameter vector θ.

Unconstrained optimization

An unconstrained problem has the form:

$$\min_{\theta} \mathcal{L}(\theta)$$

The optimization problem minimizes the loss function $\mathcal{L}(\theta)$ over every allowed value of the parameter vector θ.

At a differentiable local minimum,

$$\nabla_{\theta} \mathcal{L}(\theta) = 0$$

Unconstrained optimization

An unconstrained problem has the form:

$$\min_{\theta} \mathcal{L}(\theta)$$

The optimization problem minimizes the loss function $\mathcal{L}(\theta)$ over every allowed value of the parameter vector θ.

At a differentiable local minimum,

$$\nabla_{\theta} \mathcal{L}(\theta) = 0$$

The gradient points toward increasing loss, so gradient descent moves in the opposite direction:

$$\boxed{\theta_{k+1} = \theta_k - \eta \nabla_{\theta} \mathcal{L}(\theta_k)}$$

Unconstrained optimization

The parameter vector θ_{k+1} at optimization step k plus one equals the current parameter vector θ_k minus the learning rate η multiplied by the loss gradient evaluated at θ_k.

$$\theta_{k+1} = \theta_k - \eta \nabla_\theta \mathcal{L}(\theta_k)$$

Quick example

$$\mathcal{L}(\theta) = (\theta - 3)^2$$

Quick example

$$\mathcal{L}(\theta) = (\theta - 3)^2$$

Its derivative is

$$\frac{d\mathcal{L}}{d\theta} = 2(\theta - 3)$$

Quick example

$$\mathcal{L}(\theta) = (\theta - 3)^2$$

Its derivative is

$$\frac{d\mathcal{L}}{d\theta} = 2(\theta - 3)$$

Starting from θ_0 with $\eta=0.1$,

$$\theta_1 = 0 - 0.1(-6) = 0.6$$

Quick example

$$\mathcal{L}(\theta) = (\theta - 3)^2$$

Its derivative is

$$\frac{d\mathcal{L}}{d\theta} = 2(\theta - 3)$$

Starting from θ_0 with η=0.1,

$$\theta_1 = 0 - 0.1(-6) = 0.6$$

The update moves θ toward the minimum at θ=3.

Many optimization problems also require an equality to remain satisfied throughout the solution.

$$\mathcal{L}(\theta) = (\theta - 3)^2$$

Its derivative is

$$\frac{d\mathcal{L}}{d\theta} = 2(\theta - 3)$$

Starting from θ_0 with $\eta=0.1$,

$$\theta_1 = 0 - 0.1(-6) = 0.6$$

The update moves θ toward the minimum at $\theta=3$.

Equality constraints

Equality constraints

An equality-constrained problem has the form

$$\min_x f(x) \qquad \text{subject to} \qquad g(x) = c$$

Equality constraints

An equality-constrained problem has the form

$$\min_x f(x) \qquad \text{subject to} \qquad g(x) = c$$

Introduce a Lagrange multiplier λ and define

$$\boxed{\mathcal{L}(x, \lambda) = f(x) + \lambda\big(g(x) - c\big)}$$

Equality constraints

An equality-constrained problem has the form

$$\min_x f(x) \qquad \text{subject to} \qquad g(x) = c$$

Introduce a Lagrange multiplier λ and define

$$\boxed{\mathcal{L}(x, \lambda) = f(x) + \lambda\big(g(x) - c\big)}$$

The Lagrangian $\mathcal{L}$(x,λ) equals the objective f(x) plus the multiplier λ times the constraint error g(x) minus the required value c.

The stationary conditions are

$$\nabla_x \mathcal{L}(x, \lambda) = 0 \qquad g(x) = c$$

Equality constraints

An equality-constrained problem has the form

$$\min_x f(x) \qquad \text{subject to} \qquad g(x) = c$$

Introduce a Lagrange multiplier λ and define

$$\boxed{\mathcal{L}(x, \lambda) = f(x) + \lambda\big(g(x) - c\big)}$$

The Lagrangian $\mathcal{L}$(x,λ) equals the objective f(x) plus the multiplier λ times the constraint error g(x) minus the required value c.

The stationary conditions are

$$\nabla_x \mathcal{L}(x, \lambda) = 0 \qquad g(x) = c$$

At a constrained optimum, movement along the constraint surface cannot decrease the objective.

At a constrained optimum, movement along the constraint surface cannot decrease the objective.

This gives

$$\nabla f(x) = -\lambda \nabla g(x)$$

At a constrained optimum, movement along the constraint surface cannot decrease the objective.

This gives

$$\nabla f(x) = -\lambda \nabla g(x)$$

The two gradients are parallel at the solution.

Numerical equality-constrained example

Minimize

$$f(x, y) = x^2 + y^2$$

Numerical equality-constrained example

Minimize

$$f(x, y) = x^2 + y^2$$

subject to

$$x + y = 1$$

Numerical equality-constrained example

Minimize

$$f(x, y) = x^2 + y^2$$

subject to

$$x + y = 1$$

The Lagrangian is

$$\mathcal{L}(x, y, \lambda) = x^2 + y^2 + \lambda(x + y - 1)$$

Differentiate:

$$\frac{\partial \mathcal{L}}{\partial x} = 2x + \lambda = 0$$

$$\frac{\partial \mathcal{L}}{\partial y} = 2y + \lambda = 0$$

Numerical equality-constrained example

Minimize

$$f(x, y) = x^2 + y^2$$

subject to

$$x + y = 1$$

The Lagrangian is

$$\mathcal{L}(x, y, \lambda) = x^2 + y^2 + \lambda(x + y - 1)$$

Differentiate:

$$\frac{\partial \mathcal{L}}{\partial x} = 2x + \lambda = 0$$

$$\frac{\partial \mathcal{L}}{\partial y} = 2y + \lambda = 0$$

$$\frac{\partial \mathcal{L}}{\partial \lambda} = x + y - 1 = 0$$

Numerical equality-constrained example

Minimize

$$f(x, y) = x^2 + y^2$$

subject to

$$x + y = 1$$

The Lagrangian is

$$\mathcal{L}(x, y, \lambda) = x^2 + y^2 + \lambda(x + y - 1)$$

Differentiate:

$$\frac{\partial \mathcal{L}}{\partial x} = 2x + \lambda = 0$$

$$\frac{\partial \mathcal{L}}{\partial y} = 2y + \lambda = 0$$

$$\frac{\partial \mathcal{L}}{\partial \lambda} = x + y - 1 = 0$$

The first two equations imply

$$x = y$$

Numerical equality-constrained example

Minimize

$$f(x, y) = x^2 + y^2$$

subject to

$$x + y = 1$$

The Lagrangian is

$$\mathcal{L}(x, y, \lambda) = x^2 + y^2 + \lambda(x + y - 1)$$

Differentiate:

$$\frac{\partial \mathcal{L}}{\partial x} = 2x + \lambda = 0$$

$$\frac{\partial \mathcal{L}}{\partial y} = 2y + \lambda = 0$$

$$\frac{\partial \mathcal{L}}{\partial \lambda} = x + y - 1 = 0$$

The first two equations imply

$$x = y$$

Using the constraint,

$$2x = 1$$

Numerical equality-constrained example

Minimize

$$f(x, y) = x^2 + y^2$$

subject to

$$x + y = 1$$

The Lagrangian is

$$\mathcal{L}(x, y, \lambda) = x^2 + y^2 + \lambda(x + y - 1)$$

Differentiate:

$$\frac{\partial \mathcal{L}}{\partial x} = 2x + \lambda = 0$$

$$\frac{\partial \mathcal{L}}{\partial y} = 2y + \lambda = 0$$

$$\frac{\partial \mathcal{L}}{\partial \lambda} = x + y - 1 = 0$$

The first two equations imply

$$x = y$$

Using the constraint,

$$2x = 1$$

Therefore,

$$\boxed{x = y = \frac{1}{2}}$$

Probability normalization is exactly this type of equality constraint.

Minimize

$$f(x, y) = x^2 + y^2$$

subject to

$$x + y = 1$$

The Lagrangian is

$$\mathcal{L}(x, y, \lambda) = x^2 + y^2 + \lambda(x + y - 1)$$

Differentiate:

$$\frac{\partial \mathcal{L}}{\partial x} = 2x + \lambda = 0$$

$$\frac{\partial \mathcal{L}}{\partial y} = 2y + \lambda = 0$$

$$\frac{\partial \mathcal{L}}{\partial \lambda} = x + y - 1 = 0$$

The first two equations imply

$$x = y$$

Using the constraint,

$$2x = 1$$

Therefore,

$$\boxed{x = y = \frac{1}{2}}$$

Consider a probability vector p over K states.

Consider a probability vector p over K states.

We want the highest-entropy distribution satisfying

$$\sum_{i=1}^{K} p_i = 1$$

Consider a probability vector p over K states.

We want the highest-entropy distribution satisfying

$$\sum_{i=1}^{K} p_i = 1$$

The optimization problem is

$$\max_{p} \left[-\sum_{i=1}^{K} p_i \log p_i \right]$$

subject to normalization.

Consider a probability vector p over K states.

We want the highest-entropy distribution satisfying

$$\sum_{i=1}^{K} p_i = 1$$

The optimization problem is

$$\max_{p} \left[-\sum_{i=1}^{K} p_i \log p_i \right]$$

subject to normalization.

The Lagrangian is

$$\mathcal{L}(p, \lambda) = -\sum_{i=1}^{K} p_i \log p_i + \lambda \left(\sum_{i=1}^{K} p_i - 1 \right)$$

The Lagrangian is

$$\mathcal{L}(p, \lambda) = -\sum_{i=1}^{K} p_i \log p_i + \lambda \left(\sum_{i=1}^{K} p_i - 1 \right)$$

Differentiate with respect to p_i:

$$\frac{\partial \mathcal{L}}{\partial p_i} = -\log p_i - 1 + \lambda = 0$$

The Lagrangian is

$$\mathcal{L}(p, \lambda) = -\sum_{i=1}^{K} p_i \log p_i + \lambda \left(\sum_{i=1}^{K} p_i - 1 \right)$$

Differentiate with respect to p_i:

$$\frac{\partial \mathcal{L}}{\partial p_i} = -\log p_i - 1 + \lambda = 0$$

Therefore,

$$\log p_i = \lambda - 1$$

The Lagrangian is

$$\mathcal{L}(p, \lambda) = -\sum_{i=1}^{K} p_i \log p_i + \lambda \left(\sum_{i=1}^{K} p_i - 1 \right)$$

Differentiate with respect to p_i:

$$\frac{\partial \mathcal{L}}{\partial p_i} = -\log p_i - 1 + \lambda = 0$$

Therefore,

$$\log p_i = \lambda - 1$$

Every p_i must have the same value. Normalization then gives

$$\boxed{p_i = \frac{1}{K}}$$

The Lagrangian is

$$\mathcal{L}(p, \lambda) = -\sum_{i=1}^{K} p_i \log p_i + \lambda \left(\sum_{i=1}^{K} p_i - 1 \right)$$

Differentiate with respect to p_i:

$$\frac{\partial \mathcal{L}}{\partial p_i} = -\log p_i - 1 + \lambda = 0$$

Therefore,

$$\log p_i = \lambda - 1$$

The maximum-entropy distribution over K states assigns equal probability 1/K to every state because all stationary probabilities are equal and must sum to one.

Every p_i must have the same value. Normalization then gives

$$\boxed{p_i = \frac{1}{K}}$$

The Lagrangian is

$$\mathcal{L}(p, \lambda) = -\sum_{i=1}^{K} p_i \log p_i + \lambda \left(\sum_{i=1}^{K} p_i - 1 \right)$$

Differentiate with respect to p_i:

$$\frac{\partial \mathcal{L}}{\partial p_i} = -\log p_i - 1 + \lambda = 0$$

Therefore,

$$\log p_i = \lambda - 1$$

This proves that the uniform distribution has the largest entropy among distributions over K states.

Every p_i must have the same value. Normalization then gives

$$\boxed{p_i = \frac{1}{K}}$$

The Lagrangian is

$$\mathcal{L}(p, \lambda) = -\sum_{i=1}^{K} p_i \log p_i + \lambda \left(\sum_{i=1}^{K} p_i - 1 \right)$$

Differentiate with respect to p_i:

$$\frac{\partial \mathcal{L}}{\partial p_i} = -\log p_i - 1 + \lambda = 0$$

Therefore,

$$\log p_i = \lambda - 1$$

Normalization is one constraint. Biological design may add expected-property constraints.

Every p_i must have the same value. Normalization then gives

$$\boxed{p_i = \frac{1}{K}}$$

Maximum entropy with biological properties

Maximum entropy with biological properties

Let $h_j(x)$ measure property j of state x. Suppose the desired expected property value is c_j:

$$\mathbb{E}_{X \sim p}[h_j(X)] = c_j$$

Maximum entropy with biological properties

Let $h_j(x)$ measure property j of state x. Suppose the desired expected property value is c_j:

$$\mathbb{E}_{X \sim p}[h_j(X)] = c_j$$

The expected value of biological property $h_j(x)$ under distribution p is constrained to equal the desired target c_j.

Maximum entropy with biological properties

Let $h_j(x)$ measure property j of state x. Suppose the desired expected property value is c_j:

$$\mathbb{E}_{X \sim p}[h_j(X)] = c_j$$

The expected value of biological property $h_j(x)$ under distribution p is constrained to equal the desired target c_j. For m properties, solve:

$$\max_p H(p)$$

Maximum entropy with biological properties

Let $h_j(x)$ measure property j of state x. Suppose the desired expected property value is c_j:

$$\mathbb{E}_{X \sim p}[h_j(X)] = c_j$$

The expected value of biological property $h_j(x)$ under distribution p is constrained to equal the desired target c_j. For m properties, solve:

$$\max_p H(p)$$

Subject to:

$$\sum_x p(x) = 1$$

Maximum entropy with biological properties

Let $h_j(x)$ measure property j of state x. Suppose the desired expected property value is c_j:

$$\mathbb{E}_{X \sim p}[h_j(X)] = c_j$$

The expected value of biological property $h_j(x)$ under distribution p is constrained to equal the desired target c_j. For m properties, solve:

$$\max_p H(p)$$

Subject to:

$$\sum_x p(x) = 1 \qquad \sum_x p(x) h_j(x) = c_j \qquad \text{for } j = 1, \ldots, m.$$

Here, the Lagrangian is.

$$\begin{aligned}\mathcal{L}(p, \alpha, \lambda) = &- \sum_x p(x) \log p(x) \\ &+ \alpha \left(\sum_x p(x) - 1 \right) \\ &+ \sum_{j=1}^{m} \lambda_j \left(\sum_x p(x) h_j(x) - c_j \right).\end{aligned}$$

Here, the Lagrangian is.

$$\begin{aligned}\mathcal{L}(p, \alpha, \lambda) = &- \sum_x p(x) \log p(x) \\ &+ \alpha \left(\sum_x p(x) - 1 \right) \\ &+ \sum_{j=1}^{m} \lambda_j \left(\sum_x p(x) h_j(x) - c_j \right).\end{aligned}$$

Differentiate with respect to p(x):

$$-\log p(x) - 1 + \alpha + \sum_{j=1}^{m} \lambda_j h_j(x) = 0.$$

Here, the Lagrangian is.

$$\mathcal{L}(p, \alpha, \lambda) = -\sum_x p(x) \log p(x) + \alpha \left(\sum_x p(x) - 1 \right) + \sum_{j=1}^{m} \lambda_j \left(\sum_x p(x) h_j(x) - c_j \right).$$

The maximum-entropy distribution p*(x) assigns probability proportional to the exponential of the weighted sum of m property functions $h_j(x)$, where λ_j controls the influence of property j and Z(λ) normalizes the probabilities to sum to one.

Differentiate with respect to p(x):

$$-\log p(x) - 1 + \alpha + \sum_{j=1}^{m} \lambda_j h_j(x) = 0.$$

Solving for p(x) gives

$$\boxed{p^*(x) = \frac{1}{Z(\lambda)} \exp\left(\sum_{j=1}^{m} \lambda_j h_j(x) \right)}$$

The normalizing constant is:

$$Z(\lambda) = \sum_x \exp\left(\sum_{j=1}^m \lambda_j h_j(x)\right)$$

Here, the Lagrangian is.

$$\begin{aligned}\mathcal{L}(p, \alpha, \lambda) = &- \sum_x p(x) \log p(x) \\ &+ \alpha \left(\sum_x p(x) - 1\right) \\ &+ \sum_{j=1}^m \lambda_j \left(\sum_x p(x) h_j(x) - c_j\right).\end{aligned}$$

Differentiate with respect to p(x):

$$-\log p(x) - 1 + \alpha + \sum_{j=1}^m \lambda_j h_j(x) = 0.$$

Solving for p(x) gives

$$\boxed{p^*(x) = \frac{1}{Z(\lambda)} \exp\left(\sum_{j=1}^m \lambda_j h_j(x)\right)}$$

Here, the Lagrangian is.

$$\mathcal{L}(p, \alpha, \lambda) = -\sum_x p(x) \log p(x) + \alpha \left(\sum_x p(x) - 1 \right) + \sum_{j=1}^{m} \lambda_j \left(\sum_x p(x) h_j(x) - c_j \right).$$

The normalizing constant is:

$$Z(\lambda) = \sum_x \exp \left(\sum_{j=1}^{m} \lambda_j h_j(x) \right)$$

The partition function Z(λ) sums the unnormalized exponential property weights over every state x so that the resulting distribution is normalized.

Differentiate with respect to p(x):

$$-\log p(x) - 1 + \alpha + \sum_{j=1}^{m} \lambda_j h_j(x) = 0.$$

Solving for p(x) gives

$$\boxed{p^*(x) = \frac{1}{Z(\lambda)} \exp \left(\sum_{j=1}^{m} \lambda_j h_j(x) \right)}$$

Equality constraints require exact target values.

Equality constraints require exact target values.

Many design requirements are naturally expressed as upper or lower bounds, which leads to inequality constraints.

Inequality constraints

An inequality-constrained problem has the form:

$$\min_x f(x) \qquad \text{subject to} \qquad g(x) \leq 0.$$

Inequality constraints

An inequality-constrained problem has the form:

$$\min_x f(x) \qquad \text{subject to} \qquad g(x) \leq 0.$$

Define:

$$\mathcal{L}(x, \lambda) = f(x) + \lambda g(x), \qquad \lambda \geq 0.$$

Inequality constraints

An inequality-constrained problem has the form:

$$\min_x f(x) \qquad \text{subject to} \qquad g(x) \leq 0.$$

Define:

$$\mathcal{L}(x, \lambda) = f(x) + \lambda g(x), \qquad \lambda \geq 0.$$

The Karush-Kuhn-Tucker conditions include

$$\nabla_x \mathcal{L}(x, \lambda) = 0,$$

$$g(x) \leq 0,$$

$$\lambda \geq 0,$$

Inequality constraints

An inequality-constrained problem has the form:

$$\min_x f(x) \qquad \text{subject to} \qquad g(x) \leq 0.$$

Define:

$$\mathcal{L}(x, \lambda) = f(x) + \lambda g(x), \qquad \lambda \geq 0.$$

The Karush-Kuhn-Tucker conditions include

$$\nabla_x \mathcal{L}(x, \lambda) = 0,$$

$$g(x) \leq 0,$$

$$\lambda \geq 0,$$

and $\boxed{\lambda g(x) = 0}$.

Inequality constraints

The complementary-slackness condition λg(x)=0 means that either the inequality is inactive with g(x) strictly below zero and multiplier λ equal to zero, or the constraint is active with g(x)=0 and multiplier λ potentially positive.

An inequality-constrained problem has the form:

$$\min_x f(x) \qquad \text{subject to} \qquad g(x) \leq 0.$$

Define:

$$\mathcal{L}(x, \lambda) = f(x) + \lambda g(x), \qquad \lambda \geq 0.$$

The Karush-Kuhn-Tucker conditions include

$$\nabla_x \mathcal{L}(x, \lambda) = 0,$$

$$g(x) \leq 0,$$

$$\lambda \geq 0,$$

and $\boxed{\lambda g(x) = 0}$.

A short inequality example

Minimize

$$f(x) = (x - 3)^2$$

A short inequality example

Minimize

$$f(x) = (x - 3)^2$$

subject to

$$x \leq 1$$

A short inequality example

Minimize $f(x) = (x-3)^2$

subject to $x \leq 1$

Write the constraint as $g(x) = x - 1 \leq 0$

A short inequality example

Minimize

$$f(x) = (x-3)^2$$

subject to

$$x \leq 1$$

Write the constraint as $g(x) = x - 1 \leq 0$

The unconstrained minimum x=3 violates the constraint.

A short inequality example

Minimize

$$f(x) = (x-3)^2$$

subject to

$$x \leq 1$$

Write the constraint as $g(x) = x - 1 \leq 0$

The unconstrained minimum x=3 violates the constraint. The constrained solution lies at the boundary:

$$\boxed{x^* = 1}$$

A short inequality example

Minimize

$$f(x) = (x-3)^2$$

subject to

$$x \leq 1$$

Write the constraint as $g(x) = x - 1 \leq 0$

The unconstrained minimum x=3 violates the constraint. The constrained solution lies at the boundary:

$$\boxed{x^* = 1}$$

The Lagrangian here is:

$$\mathcal{L}(x, \lambda) = (x - 3)^2 + \lambda(x - 1)$$

The Lagrangian here is:

$$\mathcal{L}(x, \lambda) = (x - 3)^2 + \lambda(x - 1)$$

Stationarity gives

$$2(x - 3) + \lambda = 0$$

The Lagrangian here is:

$$\mathcal{L}(x, \lambda) = (x - 3)^2 + \lambda(x - 1)$$

Stationarity gives

$$2(x - 3) + \lambda = 0$$

At x =1:

$$-4 + \lambda = 0,$$

The Lagrangian here is:

$$\mathcal{L}(x, \lambda) = (x - 3)^2 + \lambda(x - 1)$$

Stationarity gives

$$2(x - 3) + \lambda = 0$$

At x =1:

$$-4 + \lambda = 0,$$

so

$$\lambda = 4$$

The Lagrangian here is:

$$\mathcal{L}(x, \lambda) = (x - 3)^2 + \lambda(x - 1)$$

Stationarity gives

$$2(x - 3) + \lambda = 0$$

At x =1:

$$-4 + \lambda = 0,$$

So

$$\lambda = 4$$

The positive multiplier measures how strongly the active constraint changes the optimum.

The Lagrangian here is:

$$\mathcal{L}(x, \lambda) = (x - 3)^2 + \lambda(x - 1)$$

Stationarity gives

$$2(x - 3) + \lambda = 0$$

At x =1:

$$-4 + \lambda = 0,$$

So

$$\lambda = 4$$

The positive multiplier measures how strongly the active constraint changes the optimum.

In biological design, for example, inequality constraints could require toxicity below a threshold, stability above a threshold, or sequence identity below a maximum.

Optimization over the probability simplex

Optimization over the probability simplex

A probability vector must satisfy

$$p_i \geq 0, \qquad \sum_i p_i = 1$$

Optimization over the probability simplex

A probability vector must satisfy

$$p_i \geq 0, \qquad \sum_i p_i = 1$$

There are several ways to maintain these constraints.

Optimization over the probability simplex

A probability vector must satisfy

$$p_i \geq 0, \qquad \sum_i p_i = 1$$

There are several ways to maintain these constraints.

One option is direct constrained optimization over the simplex.

Optimization over the probability simplex

A probability vector must satisfy

$$p_i \geq 0, \qquad \sum_i p_i = 1$$

There are several ways to maintain these constraints.

One option is direct constrained optimization over the simplex. Another option parameterizes p using logits z:

$$p_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

Optimization over the probability simplex

A probability vector must satisfy

$$p_i \geq 0, \qquad \sum_i p_i = 1$$

There are several ways to maintain these constraints.

One option is direct constrained optimization over the simplex. Another option parameterizes p using logits z:

$$p_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

The softmax parameterization guarantees that every p_i is positive and that all probabilities sum to one.

Optimization over the probability simplex

A probability vector must satisfy

$$p_i \geq 0, \qquad \sum_i p_i = 1$$

There are several ways to maintain these constraints.

One option is direct constrained optimization over the simplex. Another option parameterizes p using logits z:

$$p_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

The softmax parameterization guarantees that every p_i is positive and that all probabilities sum to one.

This converts constrained probability optimization into unconstrained optimization over the logits z.

Phew, okay, you’ve learned a lot!

Phew, okay, you've learned a lot!

- **Probability distributions**
  - A distribution specifies how probability mass is allocated across possible states.

Phew, okay, you've learned a lot!

- **Probability distributions**
  - A distribution specifies how probability mass is allocated across possible states.
- **Expectation**
  - Expectation calculates average properties, losses, and biological measurements under a distribution.

Phew, okay, you've learned a lot!

- **Probability distributions**
  - A distribution specifies how probability mass is allocated across possible states.
- **Expectation**
  - Expectation calculates average properties, losses, and biological measurements under a distribution.
- **Variance and covariance**
  - Variance records fluctuations within one quantity. Covariance records how several quantities vary together.

Phew, okay, you've learned a lot!

- **Probability distributions**
  - A distribution specifies how probability mass is allocated across possible states.
- **Expectation**
  - Expectation calculates average properties, losses, and biological measurements under a distribution.
- **Variance and covariance**
  - Variance records fluctuations within one quantity. Covariance records how several quantities vary together.
- **Conditioning and Bayes' rule**
  - Conditioning updates probabilities after observing information. Bayes' rule connects forward corruption with reverse denoising.

Phew, okay, you've learned a lot!

- **Probability distributions**
  - A distribution specifies how probability mass is allocated across possible states.
- **Expectation**
  - Expectation calculates average properties, losses, and biological measurements under a distribution.
- **Variance and covariance**
  - Variance records fluctuations within one quantity. Covariance records how several quantities vary together.
- **Conditioning and Bayes' rule**
  - Conditioning updates probabilities after observing information. Bayes' rule connects forward corruption with reverse denoising.
- **Probability simplex**
  - The simplex is the geometric space of valid discrete probability vectors.

Phew, okay, you’ve learned a lot!

- **Matrices and eigenvalues**
  - Matrices transform vectors and probability distributions. Eigenvalues determine growth, decay, stability, and stationary behavior.

Phew, okay, you've learned a lot!

- **Matrices and eigenvalues**
  - Matrices transform vectors and probability distributions. Eigenvalues determine growth, decay, stability, and stationary behavior.
- **Matrix exponential**
  - The matrix exponential accumulates infinitesimal linear changes through continuous time.

Phew, okay, you've learned a lot!

- **Matrices and eigenvalues**
  - Matrices transform vectors and probability distributions. Eigenvalues determine growth, decay, stability, and stationary behavior.
- **Matrix exponential**
  - The matrix exponential accumulates infinitesimal linear changes through continuous time.
- **Maps and pushforwards**
  - A map moves individual samples. Its pushforward moves the complete probability distribution.

Phew, okay, you've learned a lot!

- **Matrices and eigenvalues**
  - Matrices transform vectors and probability distributions. Eigenvalues determine growth, decay, stability, and stationary behavior.
- **Matrix exponential**
  - The matrix exponential accumulates infinitesimal linear changes through continuous time.
- **Maps and pushforwards**
  - A map moves individual samples. Its pushforward moves the complete probability distribution.
- **Jacobian determinant**
  - The Jacobian describes local deformation. Its determinant measures local volume change and corrects transformed densities.

Phew, okay, you've learned a lot!

- **Matrices and eigenvalues**
  - Matrices transform vectors and probability distributions. Eigenvalues determine growth, decay, stability, and stationary behavior.
- **Matrix exponential**
  - The matrix exponential accumulates infinitesimal linear changes through continuous time.
- **Maps and pushforwards**
  - A map moves individual samples. Its pushforward moves the complete probability distribution.
- **Jacobian determinant**
  - The Jacobian describes local deformation. Its determinant measures local volume change and corrects transformed densities.
- **Gradients and scores**
  - A gradient gives the direction of greatest scalar increase. A score is the gradient of log density.

Phew, okay, you’ve learned a lot!

- **Vector fields and divergence**
  - A vector field specifies local sample velocity. Divergence measures local expansion, contraction, and log-density change.

Phew, okay, you've learned a lot!

- **Vector fields and divergence**
  - A vector field specifies local sample velocity. Divergence measures local expansion, contraction, and log-density change.
- **Laplacian**
  - The Laplacian measures local curvature and generates diffusion-driven smoothing.

Phew, okay, you've learned a lot!

- **Vector fields and divergence**
  - A vector field specifies local sample velocity. Divergence measures local expansion, contraction, and log-density change.
- **Laplacian**
  - The Laplacian measures local curvature and generates diffusion-driven smoothing.
- **Entropy and KL divergence**
  - Entropy measures uncertainty. Cross-entropy measures predictive cost. KL divergence measures distributional mismatch.

Phew, okay, you've learned a lot!

- **Vector fields and divergence**
  - A vector field specifies local sample velocity. Divergence measures local expansion, contraction, and log-density change.
- **Laplacian**
  - The Laplacian measures local curvature and generates diffusion-driven smoothing.
- **Entropy and KL divergence**
  - Entropy measures uncertainty. Cross-entropy measures predictive cost. KL divergence measures distributional mismatch.
- **Optimization**
  - Optimization selects parameters or probability distributions that minimize a loss while satisfying mathematical and biological constraints.

So the **complete** generative process!

So the **complete** generative process!

Start with a simple distribution:

$$X_0 \sim p_0$$

So the **complete** generative process!

Start with a simple distribution:

$$X_0 \sim p_0$$

Construct a process that moves samples:

$$X_t = \phi_t(X_0)$$

So the **complete** generative process!

Start with a simple distribution:

$$X_0 \sim p_0$$

Construct a process that moves samples:

$$X_t = \phi_t(X_0)$$

Or

$$dX_t = b_t(X_t)\,dt + \sigma_t\,dW_t$$

So the **complete** generative process!

Start with a simple distribution:

$$X_0 \sim p_0$$

Construct a process that moves samples:

$$X_t = \phi_t(X_0)$$

Or

$$dX_t = b_t(X_t)\,dt + \sigma_t\,dW_t$$

Track the evolving distribution: $p_t = (\phi_t)_{\#}p_0$

So the **complete** generative process!

Start with a simple distribution:

$$X_0 \sim p_0$$

Construct a process that moves samples:

$$X_t = \phi_t(X_0)$$

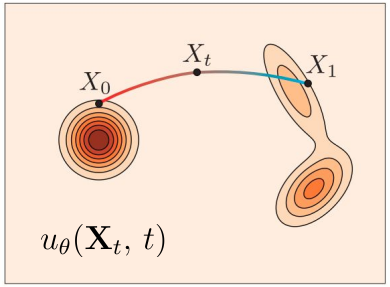
Or

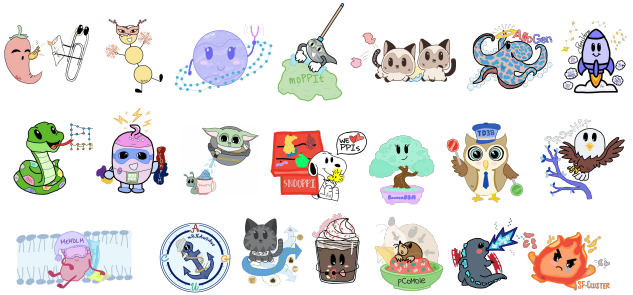
$$dX_t = b_t(X_t)\,dt + \sigma_t\,dW_t$$

Track the evolving distribution: $p_t = (\phi_t)_{\#}p_0$

Learn the unknown quantity by minimizing an expected loss:

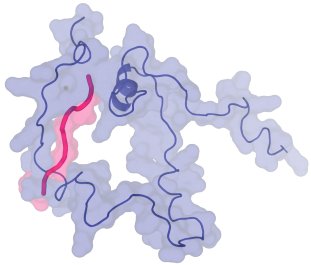
$$\theta^* = \arg\min_{\theta} \mathbb{E}\left[\ell_\theta\right]$$

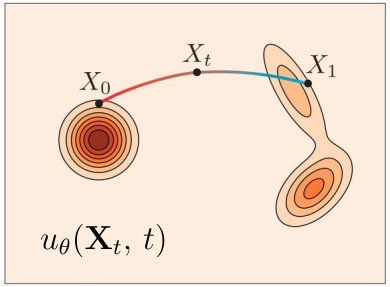


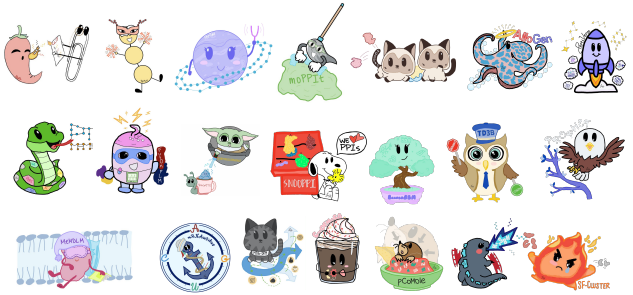


Now you have the full math foundation for this course!

Penn Engineering
University of Pennsylvania







Next, we begin generative modeling with ordinary differential equations and continuous probability transport!

Penn Engineering
University of Pennsylvania

